

NATIONAL ACADEMY OF SCIENCE AND TECHNOLOGY

Affiliated to Pokhara University

Accredited by University Grants Commission (UGC), Nepal 2022
Uttar Behadi-04, Dhangadhi, Kailali, Sudurpaschim province



Lab Report
of

“Software Engineering”

SUBMITTED BY

Name : Gajendra Awasthi
Semester : IV
Roll No : 16

SUBMITTED TO

Er. Harendra Bikram Shah
Lecturer
Department of BCA

Submission Date:2026/06/16.....

.....

TABLE OF CONTENTS

S. N.	Title	Page No.	Date	Remarks
Lab 1	Introduction to case tools for Software Engineering Designs	1	16/06/2026	
Lab 2	Designing Flowchart using case tool	4	16/06/2026	
Lab 3	Designing ER Diagram using case tool	5	16/06/2026	
Lab 4	Designing Entity relationship diagram using case tool	6	16/06/2026	
Lab 5	Designing sequence diagram using case tool	7	16/06/2026	
Lab 6	Designing use case diagram using case tool	8	16/06/2026	
Lab 7	Designing class diagram using case tool	9	16/06/2026	
Lab 8	Designing Object diagram case tool	10	16/06/2026	
Lab 9	Designing a Activity Diagram using case tool	11	16/06/2026	
Lab 10	Testing in Software Engineering	12	16/06/2026	
Lab 11	Introduction to project management tool Jira	14	16/06/2026	
Lab 12	Creating epic, story, subtask, sub-story and task and assigning it to the team members.	16	16/06/2026	
Lab 13	Team progress and reporting in Jira	21	16/06/2026	

Lab 1

Introduction to Software Engineering Tools and UML Modeling

Objective:

The objective of this lab was to become familiar with software engineering tools, install the necessary software, and gain a basic understanding of UML (Unified Modeling Language) modeling using StarUML. Additionally, we explored software engineering-related websites to enhance our learning and understanding of the field.

Tools and Resources Used

Observations

- **StarUML:** It provides an easy-to-use interface for creating UML diagrams.

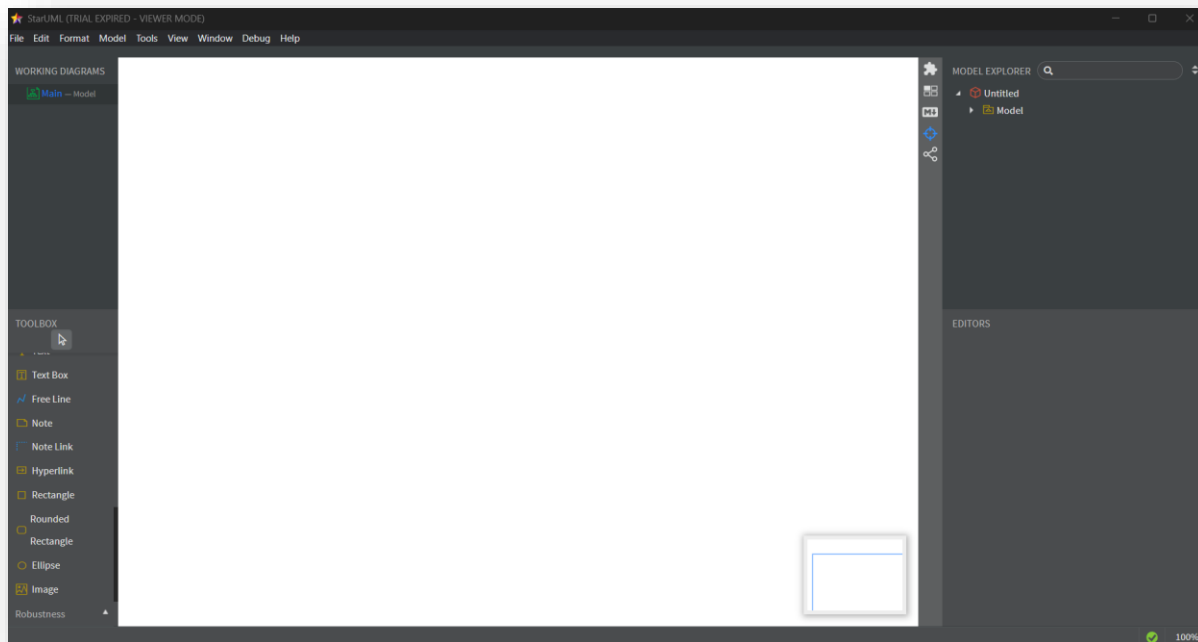


Fig 1: StarUml UI

- **Lucidchart:** It offer web-based diagramming solutions without requiring software installation.

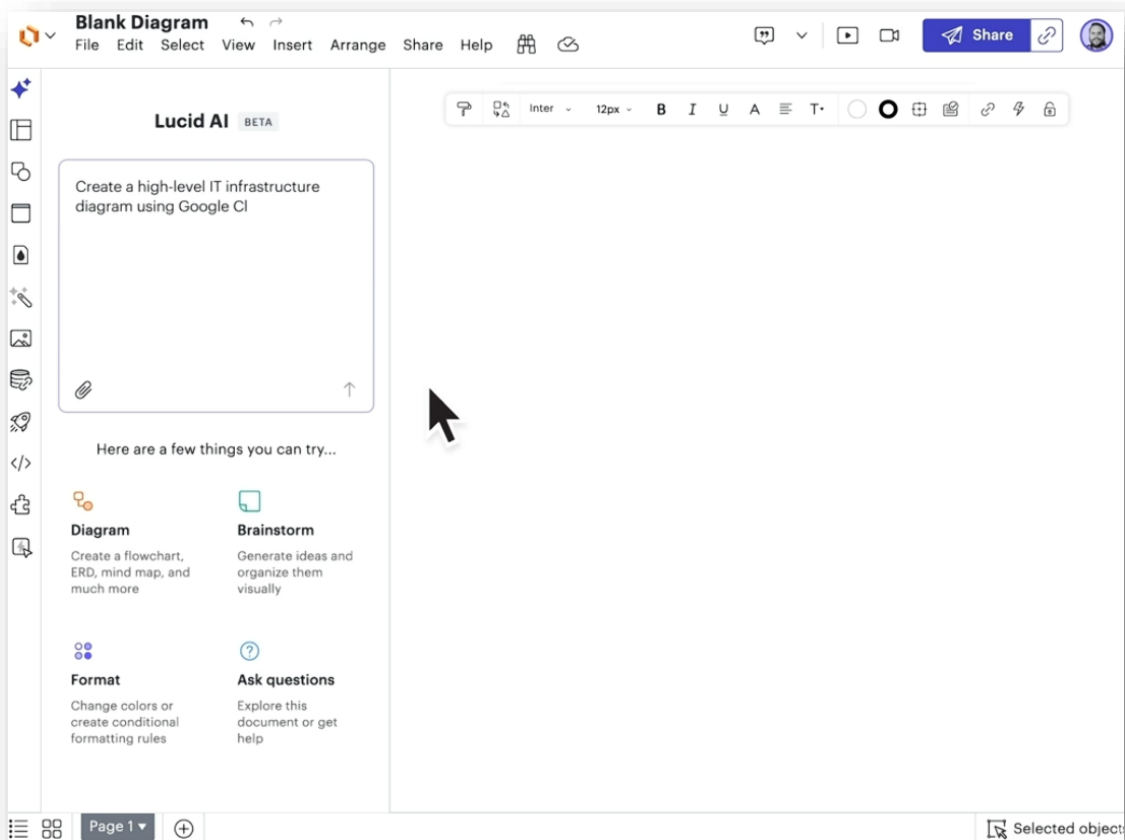


Fig 2 : Lucidchart UI

- **Draw.io:** It provides an easy-to-use drag-and-drop interface and supports cloud storage integration.

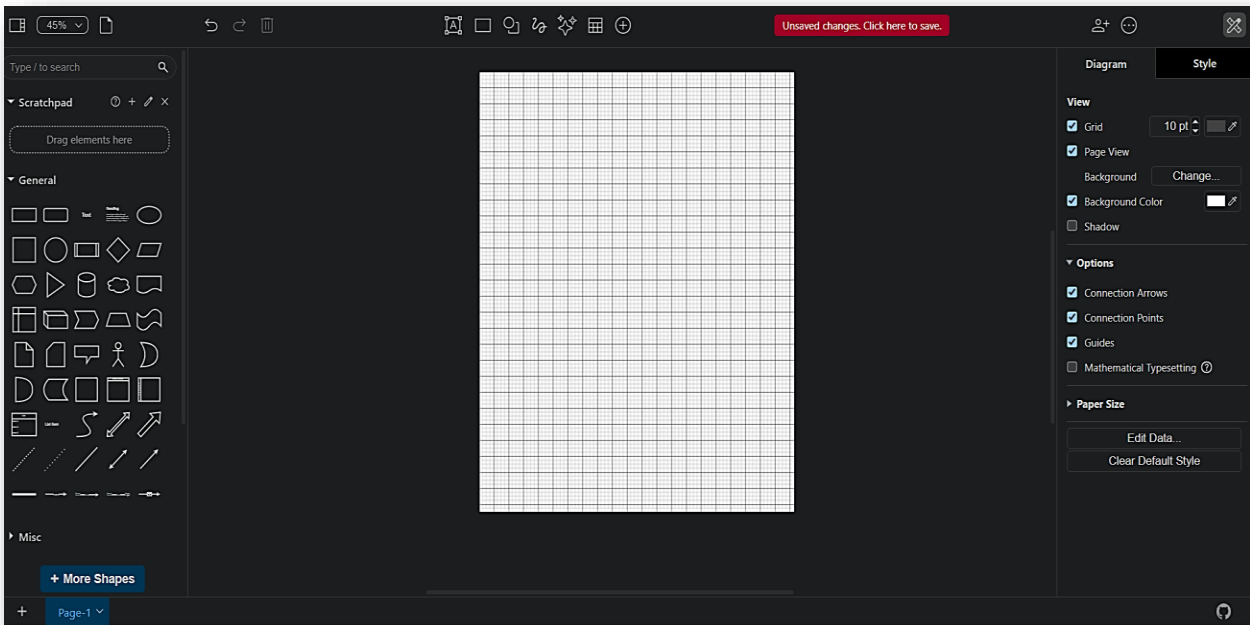


Fig 3: Draw.io UI

Lab 2:

Designing flowchart using Case tool

Theory

We learned how to create Flowchart Diagrams using a CASE (Computer-Aided Software Engineering) tool like StarUML. A flowchart is a graphical representation of the steps involved in solving a problem or completing a process. It uses standard symbols such as Start/End, Process, Input/Output, and Decision, connected with arrows to show the sequence of actions.

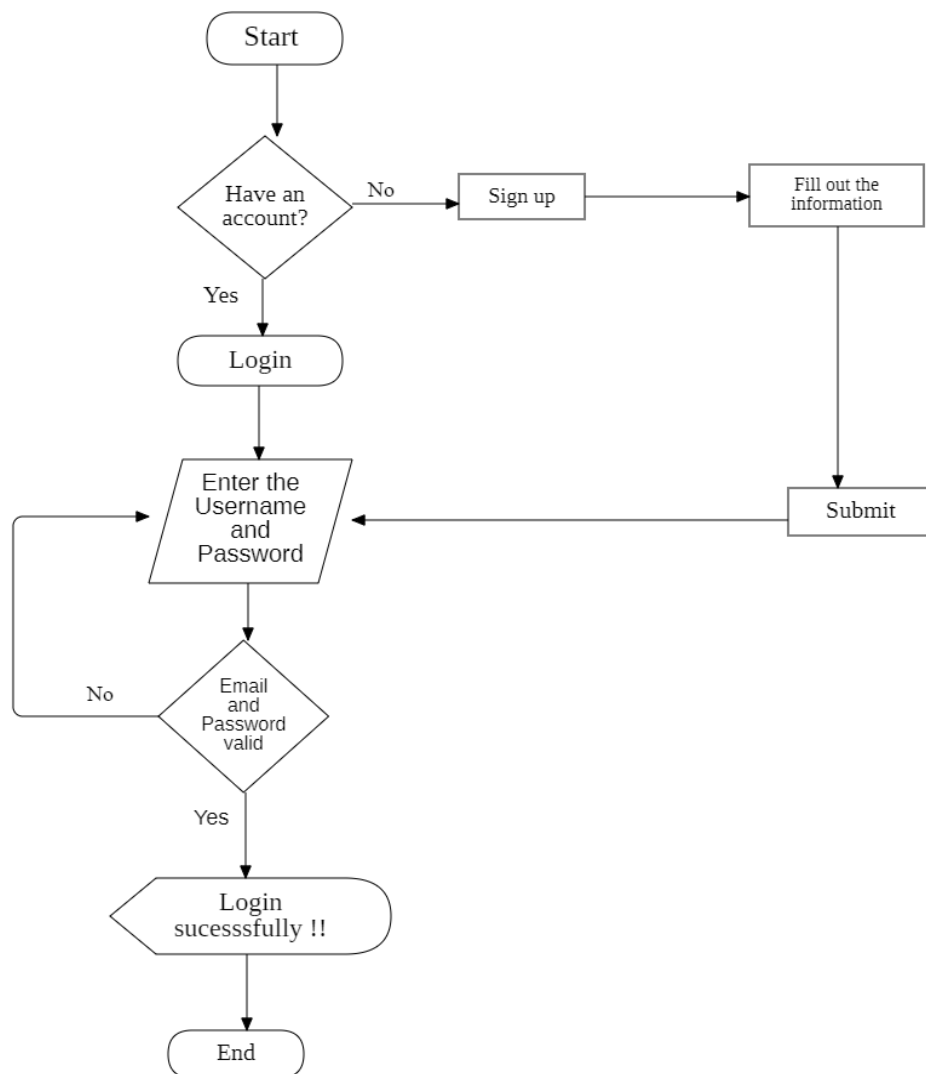


Fig 4 : Flow chart Diagram

Lab 3

Designing a Schema diagram using Case tool

Theory:

In this lab, we learned how to create a Schema Diagram using a CASE (Computer-Aided Software Engineering) tool. A Schema Diagram is a graphical representation of a database structure that shows tables, columns, data types, and relationships between tables through foreign keys. Using the CASE tool, we designed schema diagrams to convert data models into clear and accurate database designs.

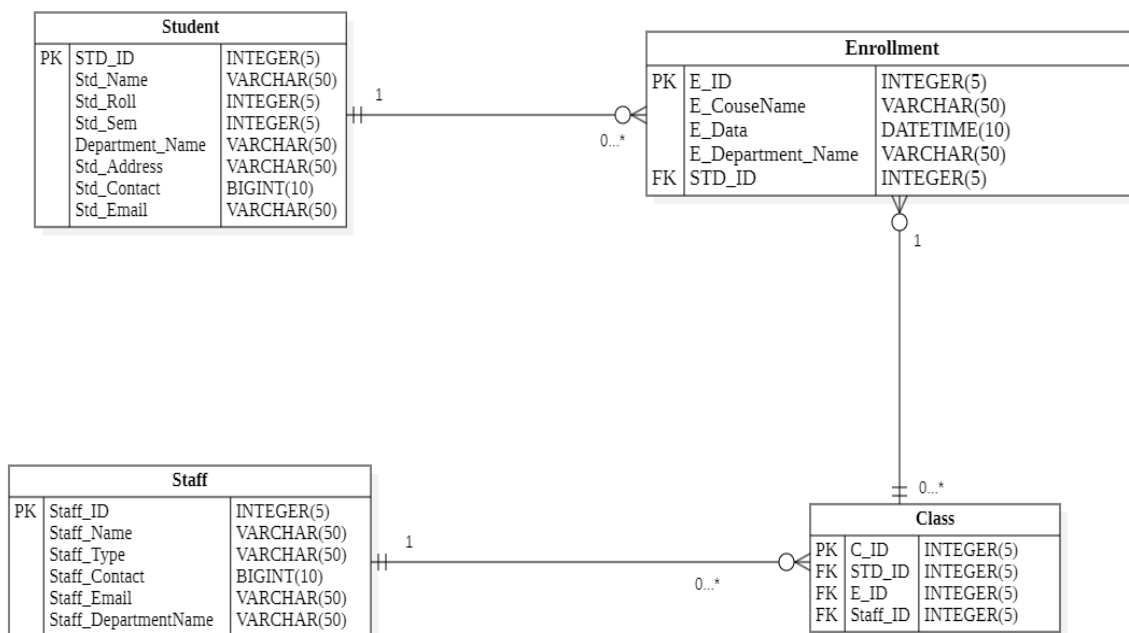


Fig 5: Schema Diagram

Lab 4

Designing ER diagram using case tool

Theory:

In this lab, we learned how to create an Entity Relationship (ER) Diagram using a CASE (Computer-Aided Software Engineering) tool. An ER Diagram is a graphical representation of the data structure of a system, showing entities, attributes, and relationships between them. Using the CASE tool, we designed ER diagrams to model database requirements clearly and accurately.

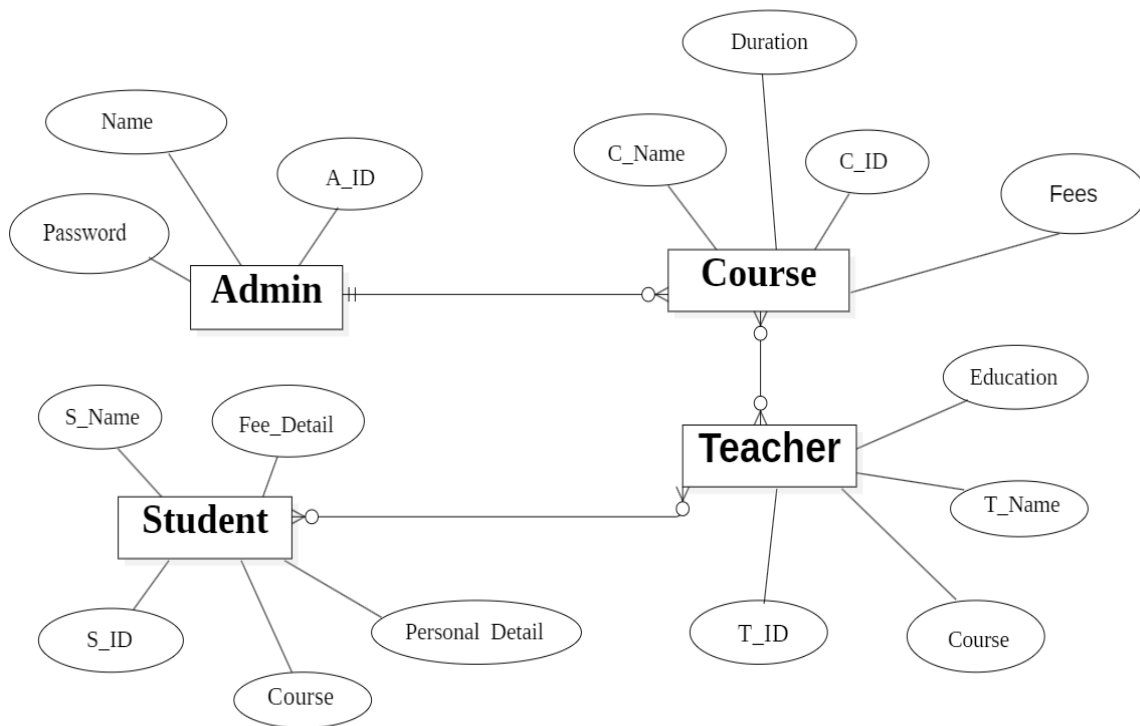


Fig 6 : ER Diagram

Lab 5

Designing a sequence diagram using Case Tool

Theory:

In this lab, we learned how to create a Sequence Diagram using a CASE (Computer-Aided Software Engineering) tool. A Sequence Diagram is a UML (Unified Modeling Language) diagram that shows how objects or system components interact over time. It represents the order of messages exchanged between actors and objects to complete a specific task or process. Using the CASE tool, we created Sequence Diagrams to understand system interactions and clearly visualize the flow of communication.

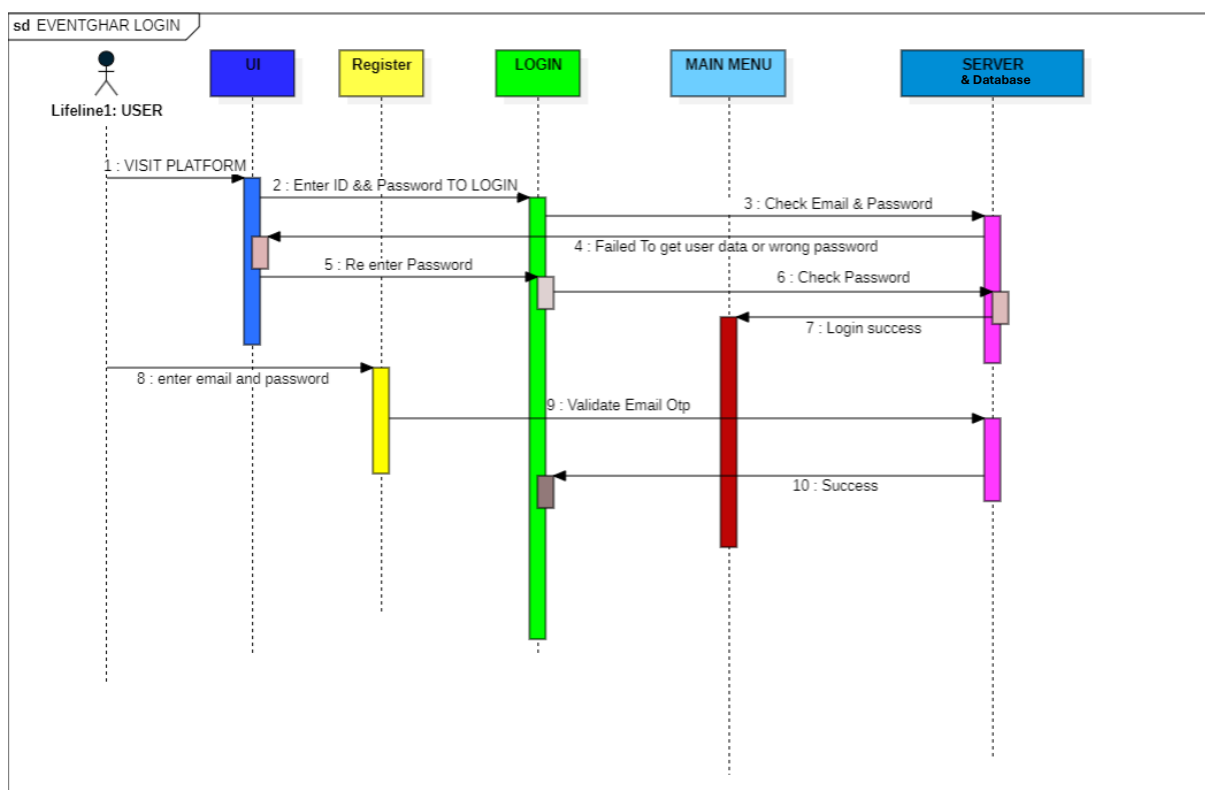


Fig 7 : Sequence Diagram

Lab 6

Designing a Use Case Diagram Using Case Tool

Theory:

In this lab, we learned how to create a Use Case Diagram using a CASE (Computer-Aided Software Engineering) tool. A Use Case Diagram is a UML diagram used to represent the functional requirements of a system by showing the interactions between users (actors) and the system. Using the CASE tool, we can identify actors, use cases, and their relationships to model system behavior clearly.

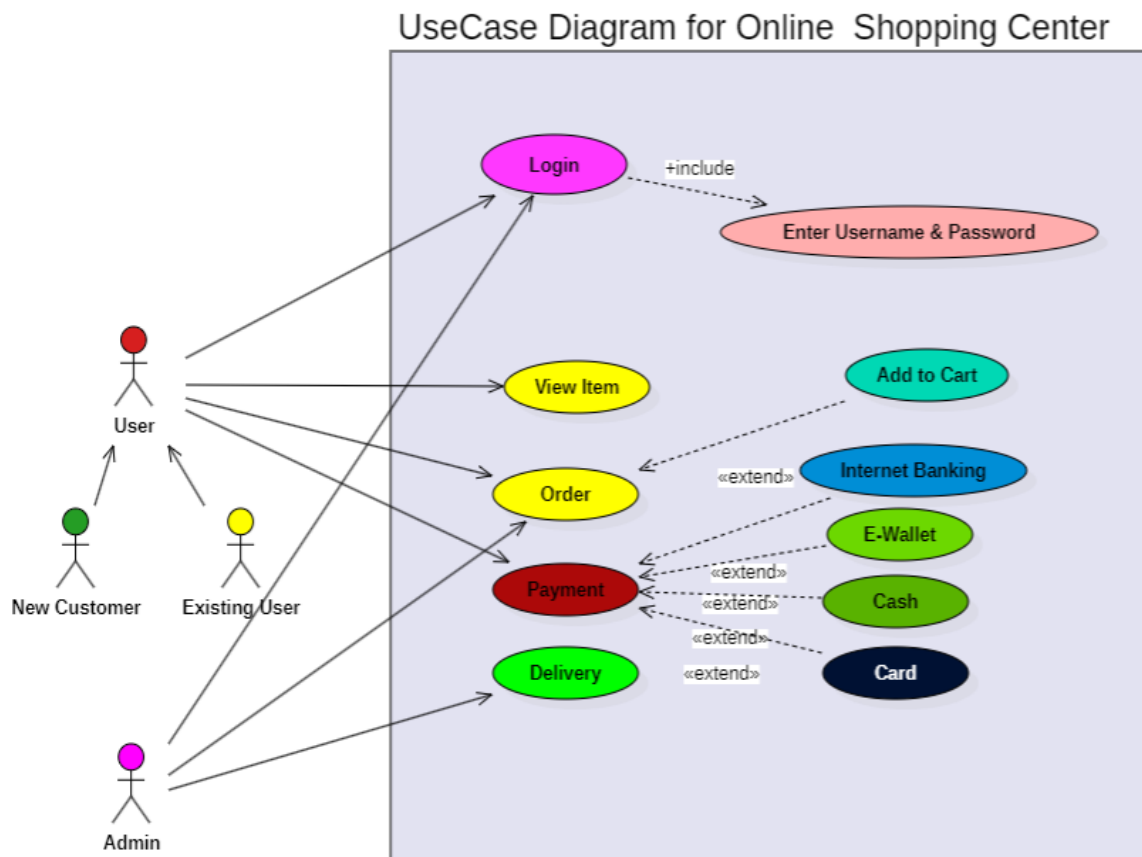


Fig 8: Use Case Diagram

Lab 7

Designing class diagram using case tool

Theory:

In this lab, we learned how to create a Class Diagram using a CASE (Computer-Aided Software Engineering) tool. A Class Diagram is a UML diagram that represents the structure of a system by showing classes, their attributes, methods, and relationships. Using the CASE tool, we designed class diagrams to model the static structure of a software system and understand how different classes interact with each other.

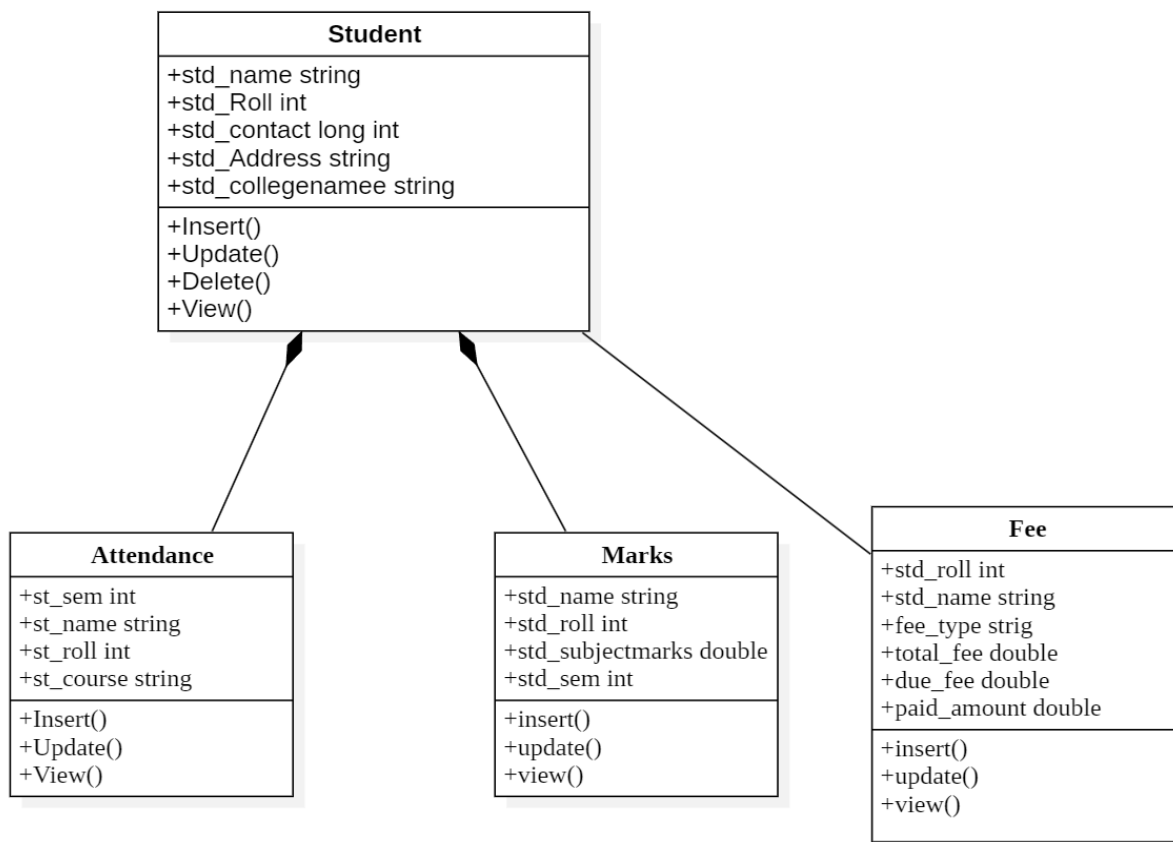


Fig 9: Class Diagram

Lab 8

Designing a Object diagram using case Tool

Theory

In this lab, we learned how to create an Object Diagram using a CASE (Computer-Aided Software Engineering) tool. An Object Diagram is a UML diagram that represents a snapshot of a system at a specific point in time by showing objects, their attribute values, and the relationships between them. Using the CASE tool, we modeled real instances of classes to better understand how objects interact within a system.

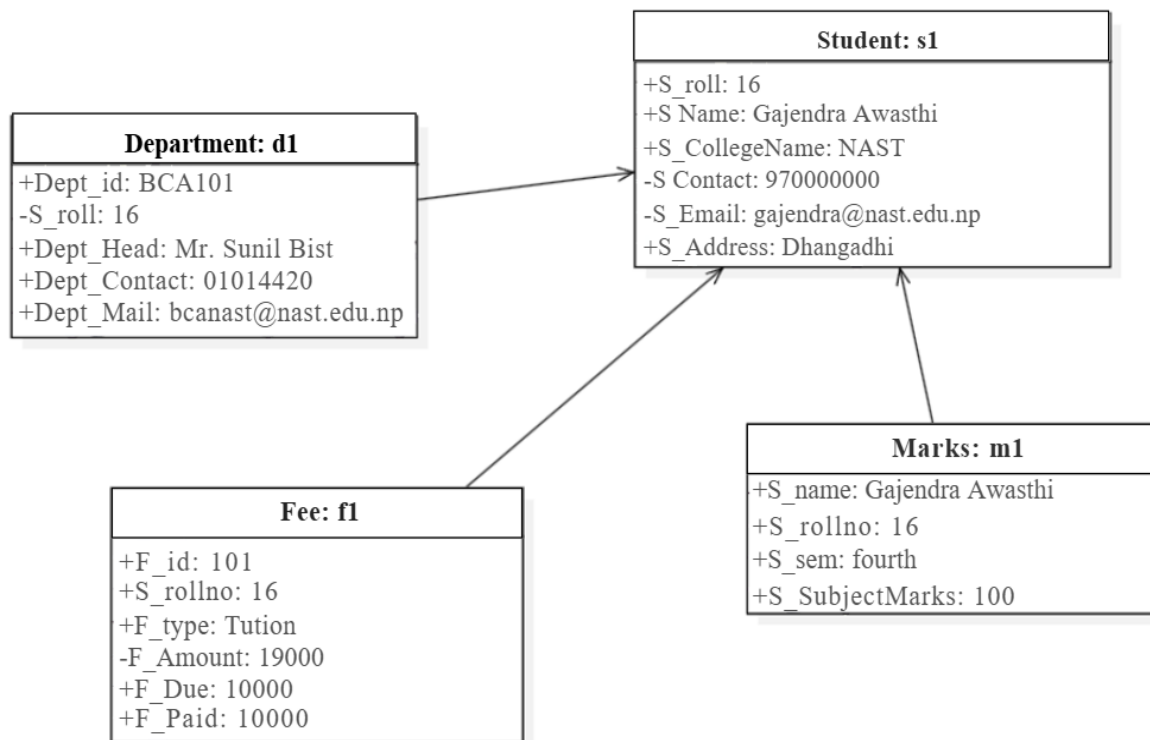


Fig 10: Object Diagram

Lab 9

Designing a Activity Diagram using case tool

Theory:

In this lab, we learned how to create an Activity Diagram using a CASE (Computer-Aided Software Engineering) tool. An Activity Diagram is a UML diagram used to represent the flow of activities and actions within a system or process. It illustrates the sequence of tasks, decision points, and parallel activities from start to finish.

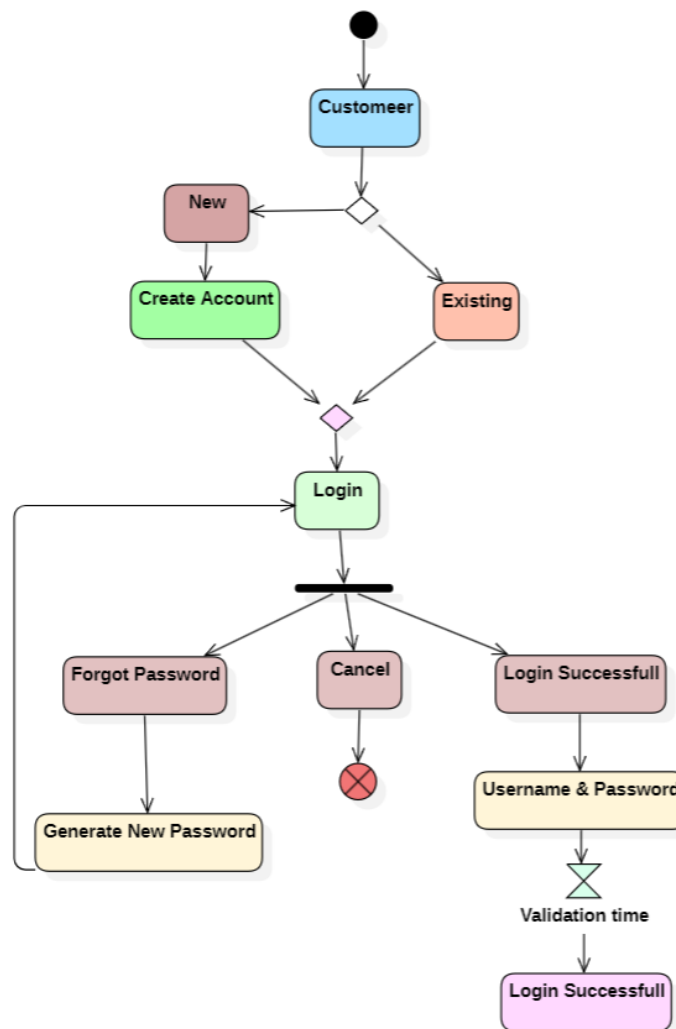


Fig 11: Activity Diagram

Lab 10

Testing in Software Engineering

Test case Development

Module: Login Module

Theory:

The purpose of this activity was to verify whether the login system functions correctly under different conditions. Various test cases were created, including valid login, wrong password, invalid user ID, duplicate user ID, forgot password, and password reset scenarios. For each test case, test steps, test data, expected output, actual output, and status were recorded to identify whether the system behaves as intended.

Test Case							
Module: Login Module							
S.no	Test Case ID	Test Scenario	Test Data	Text Steps	Expected Output	Actual Output	Status
1	TC 101	Login Validation	Name: gajendra , Email: gajendra@gmail.com , Password: Test@123	Enter user ID and Password	Login Successful	login	Pass
2	TC 102	Wrong Password	Password: test 123	Enter Password	Error message should be displayed	Error Shown	Pass
3	TC 103	Wrong User_id	usermail@gmail.com Password: Test123	Enter User id and pass	Error message should be displayed	Error not Shown	Fail
4	TC 104	Duplicate User ID	user id and Password	Enter User id	regarding wrong username	not Shown	Fail

Table 1: Test Case of login module

Test Report

Module: Login Module

Theory:

This test report presents the quality assurance and testing outcomes for the core functional components of the application, developed and evaluated during the laboratory session. The primary focus of this evaluation was the Login Module, alongside other critical subsystems including the Sign-up, Student Detail, Result Processing, Assignment, and Course Management modules.

The objective of this testing phase was to verify system stability, track defect distribution, and assess overall module readiness using structural test cases managed within Microsoft Excel. Each module was evaluated based on the total number of test cases executed against the actual passed, failed, and blocked results to calculate precise success metrics. The findings detailed in this report serve to highlight high-priority areas—ranging from stable components to urgent fixes—ensuring the application meets the required functional benchmarks before deployment.

Student Management System									
Test Report of Login Module									
S.NO	Module	Test Case ID	Pass ed	Failed	Blocked	Pass%	Fail%	Status	Tester
1	Login Module	5	4	1	0	80%	20%	fix needed	NASTEK_GAZ
2	Signup Model	10	8	2	0	80%	20%	fix needed	NASTEK_GAZ
3	Student Detail Module	10	6	4	0	60%	40%	Urgent Fix need	NASTEK_GAZ
4	Result Processing Module	10	7	2	0	70%	20%	fix needed	NASTEK_GAZ

Table 2: Test Report of login module

Lab 11

Bug Report

Theory:

During this lab, we performed software testing and documented the defects identified in different modules of the application. The bug report records important details such as Bug ID, module name, test case, bug description, severity level, status, and remarks. A total of eight bugs were reported across modules including Login, Registration, Result Processing, Report, Assignment, Attendance, Course, and Notice. The report helps track issues systematically, prioritize fixes based on severity, and improve the overall quality and reliability of the software before deployment.

BUG REPORT ON COLLEGE MANAGEMENT SYSTEM								
S.NO	BUG_ID	MODULE	TEST CASE ID	BUG DISCRPTION	SEVERITY	STATUS	REMARK	TESTER
1	B101	LOGIN	TC101	ERROR MESSAGE NOT DISPLAYED	MEDIUM	OPEN	PROPER MESSAGE SHOULD BE DISPLAYED ACCORDING	NASTEK_GAZ
2	B102	REGISTRATION	TC102	DUPLICATE DATA ENTRY	HIGH	OPEN	DUPLICATE DATA ENTRY SHOULD NOT BE THERE	NASTEK_GAZ
3	B103	RESULT PROCESSING	TC103	PROPER RANK IS NOT SHOWING	HIGH	OPEN	RANK ALGORITHM SHOULD BE UPDATE	NASTEK_GAZ
4	B104	REPORT	TC104	ALL DETAILS ARE NOT	LOW	CLOSED	ALREADY FIXED	NASTEK_GAZ
5	B105	ATTENDANCE MODULE	TC105	ATTENDANCE NOT SAVING	HIGH	OPEN	ATTENDANCE DATA SHOULD BE SAVED	NASTEK_GAZ
6	B106	SEARCH MODULE	TC106	SEARCH RETURNS INCORRECT RESULTS	MEDIUM	OPEN	SEARCH SHOULD RETURN RELEVANT RESULTS	NASTEK_GAZ
7	B107	ADMIN PANEL	TC107	USER DELETE BUTTON NOT WORKING	HIGH	OPEN	ADMIN SHOULD BE ABLE TO DELETE USERS	NASTEK_GAZ
8	B108	EXAM SCHEDULER	TC108	EXAM DATE NOT DISPLAYED	LOW	CLOSED	ALREADY FIXED	NASTEK_GAZ

Table 3: Bug Report on College Management System

Introduction to project management tool Jira

Theory:

Jira is an agile project management tool used by teams to plan, track, release, and support software development with confidence. It serves as a single source of truth for the entire development lifecycle, helping autonomous teams stay aligned with business goals while working efficiently. Whether used for simple projects or DevOps practices, Jira makes it easy to move work forward, stay organized, and communicate effectively in context.

Jira supports various agile project management methodologies, including Scrum and Kanban. It provides tools for agile planning, customizable boards, estimating work, and generating reports to measure progress and team velocity, with workflows that can be adapted to different frameworks.

Key terms in Jira:

1. **Epic:** An epic is a large goal or feature that contains smaller pieces of work (e.g., building a login system).
2. **Story:** A story is a feature or requirement described from a user's point of view (e.g., "As a user, I want to log in.").
3. **Task:** A task is a specific technical work item that needs to be completed (e.g., "Build login page UI").
4. **Sub-task:** A sub-task is a smaller step or activity within a task (e.g., "Add validation to form fields").
5. **Sprint:** A sprint is a fixed time period (usually 1–2 weeks) during which a team works on a set of planned tasks.

Conclusion:

Learning to use Jira equips us with an essential skill for managing modern projects. It adds structure to teamwork, helps ensure that no task is missed, and provides a clear view of project progress. Whether for small team assignments or large-scale software development, Jira makes task management simpler, more efficient, and more collaborative.

Lab 12

Creating Epic, Story, Sub-task, Sub-story and Task and Assigning it to the team members in Jira

Theory:

The goal of this lab session was to learn how to use Jira for effective project management. We were introduced to key concepts such as epics, stories, tasks, and sub-tasks, as well as how to form teams and assign work in an agile development environment.

In today's lab session, we further explored Jira as a project management tool used for managing software development projects.

What we learned today:

1. Creating a Project:

We began by creating a new project in Jira. We used the Scrum template, which is ideal for agile teams. This gave us a board where we could visualize and manage our workflow.

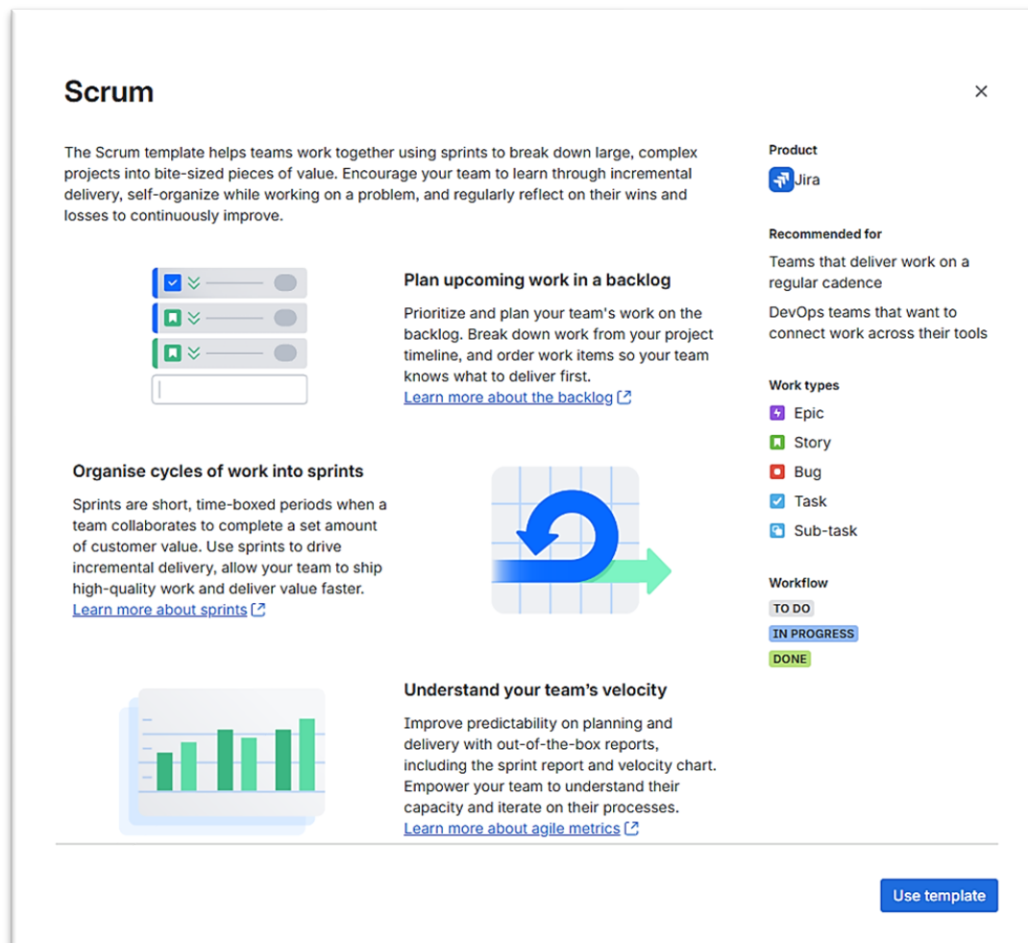


Fig 12: Creating Scrum project in Jira

2. Creating Epics:

An epic is like a big goal or feature. For example, in a Student Information System, an epic could be "Login Module". It covers all related work under one umbrella.



Fig 13: Creating Epics in Jira

3. Creating Stories:

Each epic was broken into stories. Stories are more specific and user-focused. For example, "As a user, I want to log in so I can access my dashboard." These help us understand what the end-user needs.

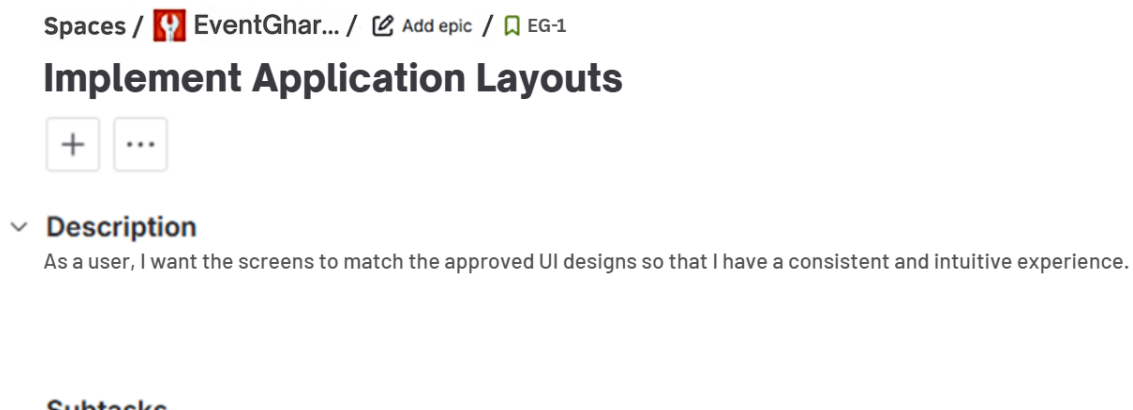


Fig 14: Creating Stories in Jira

4. Creating Tasks and Sub-tasks:

Stories were then divided into tasks, like "Design login page," "Create backend API," etc. Some tasks were further divided into sub-tasks for more detailed planning.

Spaces /  EventGhar... /  EG-1 /  EG-1

Start Designing UI & then Code IT

Description
[Edit description](#)

▼ **Subtasks** ...  + 0% Done

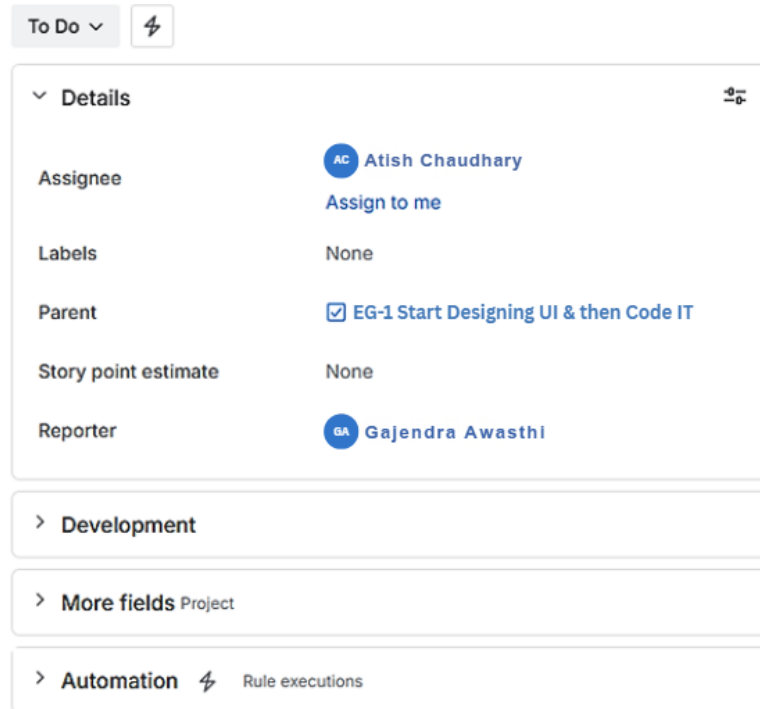
Work	Priority	Assignee	Status
 EG-1 Convert UI Designs into Source code	Medium	 Unassigned	TO DO ▼
 EG-1 Finalize UI Design	Medium	 Unassigned	TO DO ▼

 Subtask ▼ 

Fig 15: Creating Tasks & Subtasks in Jira

5. Assigning Work and Collaborating:

We learned how to assign each item to a team member. This part made Jira feel very collaborative. Everyone knew what they were responsible for, and it helped us to collaborate virtually.



The screenshot displays the 'To Do' section of a Jira issue. At the top, there is a 'To Do' dropdown menu and a lightning bolt icon. Below this, the 'Details' section is expanded, showing the following information:

- Assignee:** Atish Chaudhary (AC) with a link to 'Assign to me'.
- Labels:** None
- Parent:** EG-1 Start Designing UI & then Code IT (checked)
- Story point estimate:** None
- Reporter:** Gajendra Awasthi (GA)

Below the details section, there are three expandable sections:

- Development**
- More fields** Project
- Automation** ⚡ Rule executions

Fig 16: Assigning work in Jira

6. Using Boards to Track Progress:

The Jira board showed us where every task stood - whether it was "To Do," "In Progress," or "Done." This made it easy to track our progress.

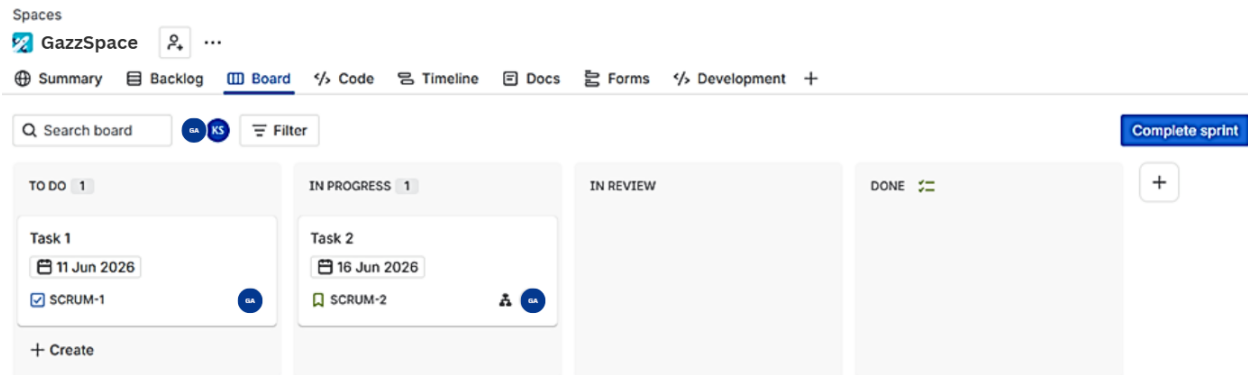


Fig 17: Track Task Progress in Jira

Conclusion:

This lab session helped me to understand the practical use of Jira in managing projects. It showed how big tasks can be broken down and assigned, how progress can be tracked visually, and how teams can collaborate better. Exploring Jira myself during the lab helped me get a better understanding. I look forward to using it more in future projects.

Lab 13

Team progress and reporting in Jira

Theory:

Today in the lab, our team focused on completing assigned tasks and tracking progress through Jira. We updated the status of completed work and logged ongoing activities. Jira was used to ensure transparency, maintain task accountability, and keep the project progress aligned with the planned schedule. This reporting process helped the team stay organized and facilitated effective collaboration among team members.

Progress Tracking Mechanisms The core of tracking progress involved continuously updating the status of issues and utilizing Jira's board features:

I. Board View:

This view provided a detailed, column-based representation of the current workflow status of issues. The board was organized into the following columns: TO DO, IN PROGRESS, TESTED, and DONE. Issues were distributed across these columns, reflecting their real-time progress. Specifically:

- **To Do Column:** Displays tasks that have not yet been started.
- **In Progress Column:** Shows tasks currently being worked on by team members.
- **Tested Column:** Contains tasks that have been tested successfully.
- **Done Column:** Includes tasks that have been completed successfully.
- **In Review Column:** Lists tasks waiting for review, testing, or approval.

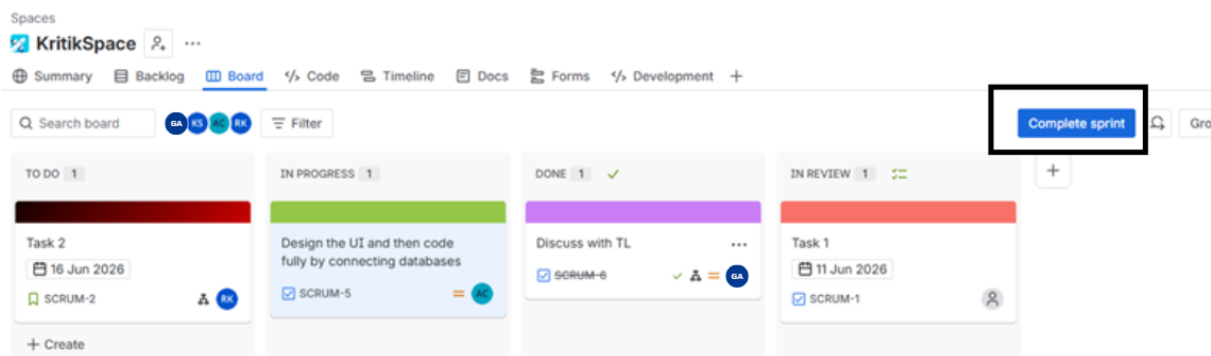


Fig 18: Board view in Jira

II. Backlog View:

The Jira Backlog displays all tasks planned for the current sprint. In this project, there are four work items: Student Registration Management, Student Enrollment Management, Attendance Tracking and Student Grade Management. Each task has an assigned team member and a status such as To Do or In Review. The backlog helps the team prioritize work, organize sprint tasks, and track progress before tasks are moved to the active board.

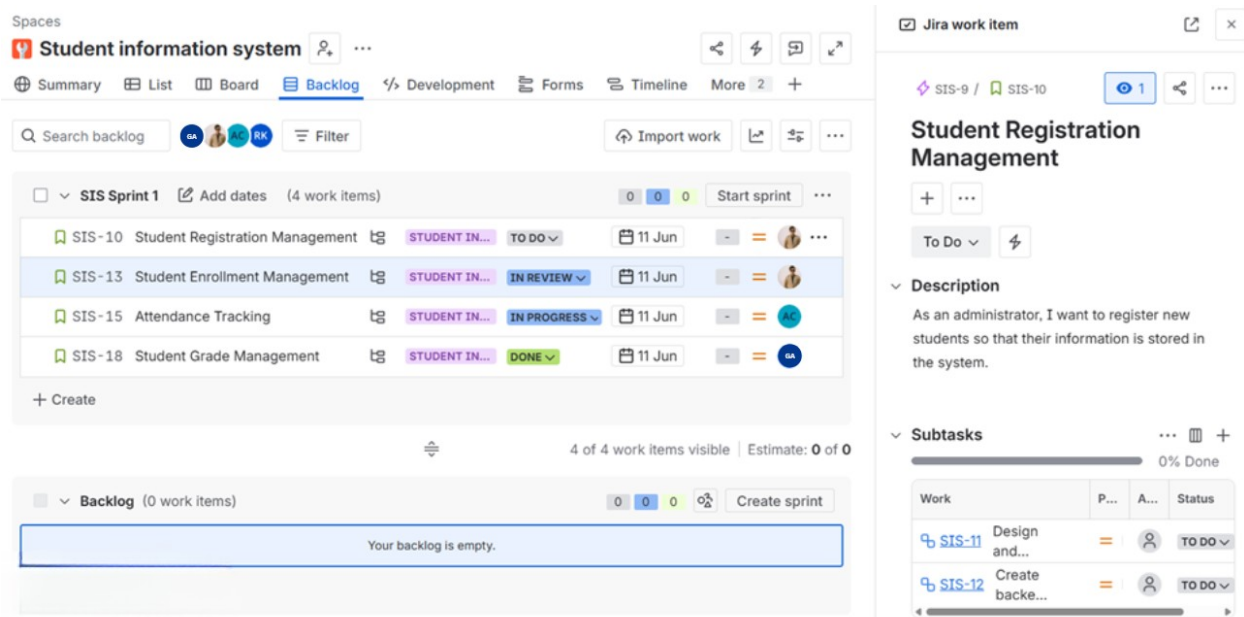


Fig 19: Backlog view in Jira

III. Summary View:

In Jira Summary provides a high-level overview of a project's work and activity. It displays key metrics such as completed, created, and updated issues, upcoming due dates, task status distribution, priority breakdown, and recent team activity. This dashboard helps users quickly understand project progress, track workload, and monitor ongoing work without needing to open individual issues.

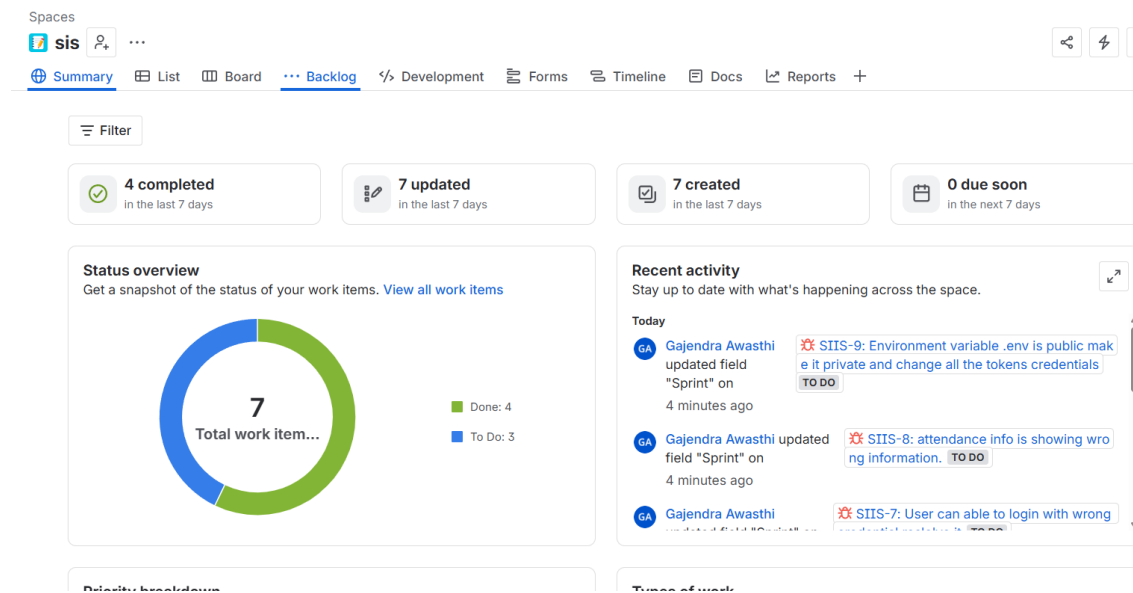


Fig 20: Summary view in Jira